```
  return (n) => {                                                    #5
    if (n < 0) {
      throw new RangeError('n can not be negative');
    } else if (n > 1) {
      return get(n - 1) + get(n - 2);                                #6
    }
    return 1;                                                        #7
  };
})();                                                                #8
```

#1 We use IIFE to create a closure for memoization
#2 This is our "history". Initialized with return values for 1 and 0
#3 This function (defined using lambda notation) takes care of memoization
#4 Update history with a true recursive call
#5 We return a function (which will be assigned to `const fibonacci`), effectively computing the `n`-th Fibonacci number, for input `n`.
#6 Computes `f(n)` by retrieving values for `n-1` and `n-2`
#7 Covers the case n === 1 || n === 0
#8 Executes the IIFE so the result of the function call is assigned to the constant on the first line

Both using naive recursion and memoization can lead to space issues and out-of-memory exceptions (more in a few sections).

E.1.2 Good Recursion

The previous example shows that just using recursion didn’t really improve the situation compared to an iterative